Reinforcement Learning

Dr Stephen James

Autumn Term 2026

Imperial College London

Reinforcement Learning
Lecture 4: Temporal Difference Methods

Dr Stephen James

Autumn Term 2026

Imperial College London

Outline

1. Introduction
2. TD(0) for Policy Evaluation
3. From Prediction to Control
4. SARSA: On-Policy TD Control
5. Q-Learning: Off-Policy TD Control
6. Expected SARSA
7. Double Learning

Introduction

1. Introduction

2. TD(0) for Policy Evaluation

3. From Prediction to Control

4. SARSA: On-Policy TD Control

5. Q-Learning: Off-Policy TD Control

6. Expected SARSA

7. Double Learning

From Monte Carlo to Temporal Difference Learning

Week 3: Monte Carlo - learn from complete episodes



Week 4: TD Learning - learn from every step

Today's Journey

- Combine the best of Monte Carlo and Dynamic Programming
- Learn online from incomplete episodes
- Master TD(0), SARSA, and Q-learning
- Understand the bias-variance tradeoff
- See the path to modern deep RL algorithms

What was Monte Carlo's biggest limitation?

Monte Carlo Problem: Must wait until episode ends to update

Example: Blackjack

- Play entire hand (could be 4+ actions)
- Only learn when hand finishes
- No learning during the game
- What if episodes are very long? Or never end?

Can we learn online, step by step, without waiting for episodes to complete?

Can we combine the best of Monte Carlo and Dynamic Programming?

What was Monte Carlo's biggest limitation?

Monte Carlo Problem: Must wait until episode ends to update

Example: Blackjack

- Play entire hand (could be 4+ actions)
- Only learn when hand finishes
- No learning during the game
- What if episodes are very long? Or never end?

Can we learn online, step by step, without waiting for episodes to complete?

Can we combine the best of Monte Carlo and Dynamic Programming?

TD Learning: The Best of Both Worlds

Definition (Temporal Difference (TD) Learning)
A class of model-free methods that learn value functions by bootstrapping from current estimates, updating based on the difference between successive predictions.

Core idea: New Estimate $=$ Old Estimate $+$ StepSize $\times$ [Target $-$ Old Estimate]

Key Characteristics:

- **Temporal**: Learn from differences across time steps
- **Bootstrapping**: Use current estimates to improve estimates
- **Sample-based**: Learn from experience, not models
- **Online**: Update after every transition

TD Learning: The Best of Both Worlds

Definition (Temporal Difference (TD) Learning)
A class of model-free methods that learn value functions by bootstrapping from current estimates, updating based on the difference between successive predictions.

Core idea: New Estimate $=$ Old Estimate $+$ StepSize $\times$ [Target $-$ Old Estimate]

Key Characteristics:

- **Temporal**: Learn from differences across time steps
- **Bootstrapping**: Use current estimates to improve estimates
- **Sample-based**: Learn from experience, not models
- **Online**: Update after every transition

TD(0) for Policy Evaluation

1. Introduction

2. TD(0) for Policy Evaluation

3. From Prediction to Control

4. SARSA: On-Policy TD Control

5. Q-Learning: Off-Policy TD Control

6. Expected SARSA

7. Double Learning

TD(0): The Simplest TD Method

Definition (TD Error)
The TD error measures how wrong our current prediction was:

$$\delta_t = [R_{t+1} + \gamma V(S_{t+1})] - V(S_t)$$

Interpretation:

- If $\delta_t > 0$: We underestimated $V(S_t)$, increase it
- If $\delta_t < 0$: We overestimated $V(S_t)$, decrease it
- If $\delta_t = 0$: Our estimate was perfect (locally)

TD(0) update becomes: $V(S_t) \leftarrow V(S_t) + \alpha[\underbrace{R_{t+1} + \gamma V(S_{t+1})}_{\text{TD target}} - V(S_t)]$

TD(0): The Simplest TD Method

Definition (TD Error)
The TD error measures how wrong our current prediction was:

$$\delta_t = [R_{t+1} + \gamma V(S_{t+1})] - V(S_t)$$

Interpretation:

- If $\delta_t > 0$: We underestimated $V(S_t)$, increase it
- If $\delta_t < 0$: We overestimated $V(S_t)$, decrease it
- If $\delta_t = 0$: Our estimate was perfect (locally)

TD(0) update becomes: $V(S_t) \leftarrow V(S_t) + \alpha[\underbrace{R_{t+1} + \gamma V(S_{t+1})}_{\text{TD target}} - V(S_t)]$

TD(0): The Simplest TD Method

Definition (TD Error)
The TD error measures how wrong our current prediction was:

$$\delta_t = [R_{t+1} + \gamma V(S_{t+1})] - V(S_t)$$

Interpretation:

- If $\delta_t > 0$: We underestimated $V(S_t)$, increase it
- If $\delta_t < 0$: We overestimated $V(S_t)$, decrease it
- If $\delta_t = 0$: Our estimate was perfect (locally)

TD(0) update becomes: $V(S_t) \leftarrow V(S_t) + \alpha[\underbrace{R_{t+1} + \gamma V(S_{t+1})}_{\text{TD target}} - V(S_t)]$

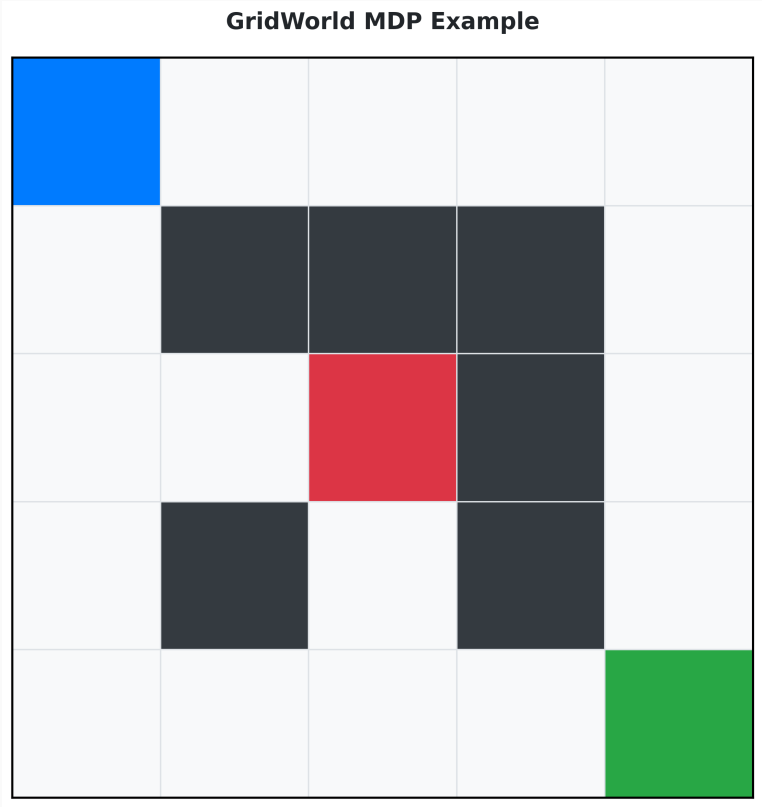
How do we implement TD(0) for Policy Evaluation?

Tabular TD(0) for estimating V^π

Input: Policy π to be evaluated
Algorithm parameter: Step size $\alpha \in (0, 1]$
Initialize: $V(s)$ for all $s \in \mathcal{S}$, arbitrarily except $V(\text{terminal}) = 0$

repeat
    Initialize S (i.e. env.reset())
    repeat for each step of episode
        $A \leftarrow$ action given by π for S
        Take action A, observe R, S' (i.e. env.step())
        $V(S) \leftarrow V(S) + \alpha[R + \gamma V(S') - V(S)]$
        $S \leftarrow S'$
    until S is terminal
until converged

Example: TD(0) policy evaluation in action on GridWorld



Same 5×5 GridWorld as in the MC lecture

MDP Setup: Goal=+1, Trap=-1, Step=0, $\gamma = 0.9$

Policy to Evaluate: A fixed policy that always goes right, then down (along the top row, then down the right column to the goal).

TD(0) settings:

- Initialise $V(s) = 0$ for all s
- Step size $\alpha = 0.5$
- Update after every step, not at the end of the episode

TD(0) Episode 1

$$V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$



$V(s)$ **after episode 1**

0.00	0.00	0.00	0.00	0.00
0.00				0.00
0.00	0.00	Trap		0.00
0.00		0.00		0.50
0.00	0.00	0.00	0.00	Goal

Every step except the last has TD error 0: $R = 0$ and $V(S') = 0 = V(S)$.
Final step: $V(3,4) \leftarrow 0 + 0.5[1 + 0 - 0] = 0.5$.
Only one state is updated (MC would update all 8 visited states).

TD(0) Episode 1

$$V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$



$V(s)$ **after episode 1**

0.00	0.00	0.00	0.00	0.00
0.00				0.00
0.00	0.00	Trap		0.00
0.00		0.00		0.50
0.00	0.00	0.00	0.00	Goal

Every step except the last has TD error 0: $R = 0$ and $V(S') = 0 = V(S)$.
Final step: $V(3,4) \leftarrow 0 + 0.5[1 + 0 - 0] = 0.5$.
Only one state is updated (MC would update all 8 visited states).

TD(0) Episode 2

$$V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$



$V(s)$ **after episode 2**

0.00	0.00	0.00	0.00	0.00
0.00				0.00
0.00	0.00	Trap		0.225
0.00		0.00		0.75
0.00	0.00	0.00	0.00	Goal

Same path again. Updating online along the path:

$V(2,4) \leftarrow 0 + 0.5[0 + 0.9 \cdot \mathbf{0.5} - 0] = 0.225$ (bootstraps off $V(3,4)$)

$V(3,4) \leftarrow 0.5 + 0.5[1 + 0 - 0.5] = 0.75.$

TD(0) Episode 2

$$V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$



$V(s)$ **after episode 2**

0.00	0.00	0.00	0.00	0.00
0.00				0.00
0.00	0.00	Trap		0.225
0.00		0.00		0.75
0.00	0.00	0.00	0.00	Goal

Same path again. Updating online along the path:
$V(2,4) \leftarrow 0 + 0.5[0 + 0.9 \cdot \mathbf{0.5} - 0] = 0.225$ (bootstraps off $V(3,4)$)
$V(3,4) \leftarrow 0.5 + 0.5[1 + 0 - 0.5] = 0.75.$

TD(0) Episode 3

$$V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$



$V(s)$ **after episode 3**

0.00	0.00	0.00	0.00	0.00
0.00				0.101
0.00	0.00	Trap		0.45
0.00		0.00		0.875
0.00	0.00	0.00	0.00	Goal

Same path again. Value moves back one more step each episode:

$V(1,4) \leftarrow 0 + 0.5[0 + 0.9 \cdot 0.225 - 0] \approx 0.10,$

$V(2,4) \leftarrow 0.225 + 0.5[0 + 0.9 \cdot 0.75 - 0.225] = 0.45, \quad V(3,4) \leftarrow 0.875.$

TD(0) Episode 3

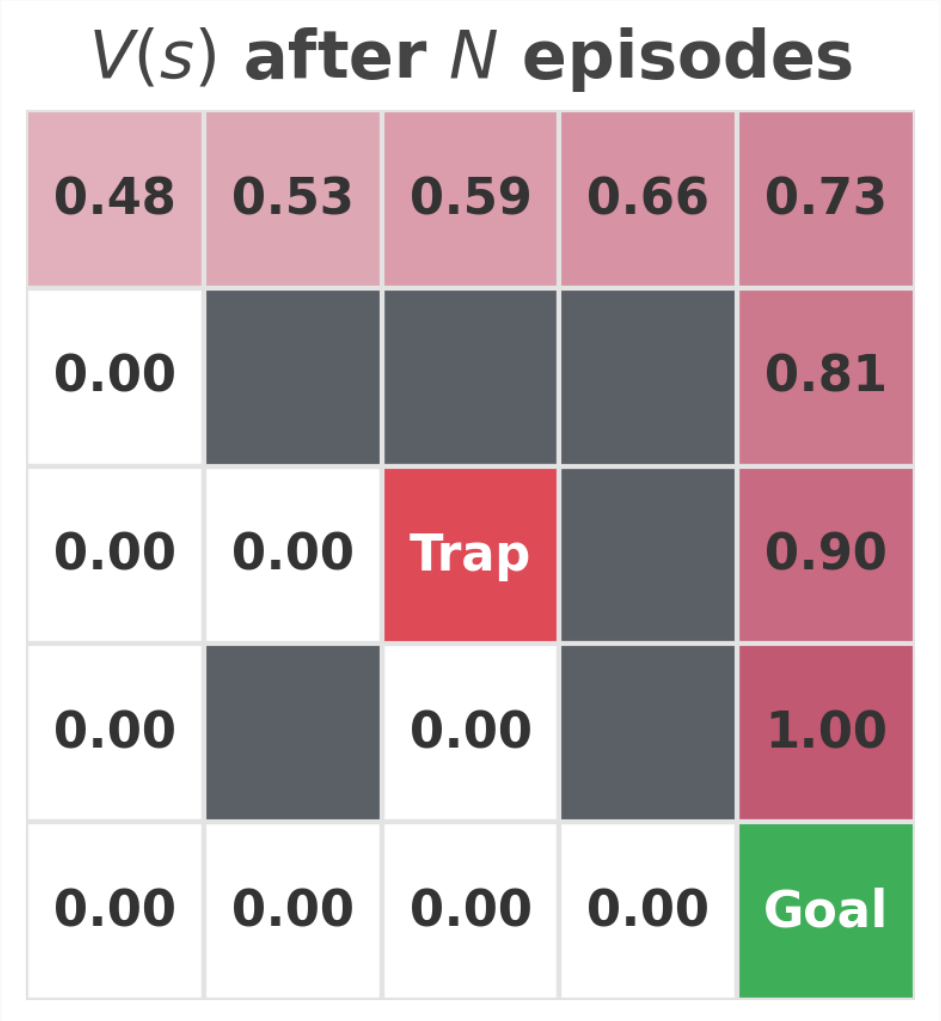
$$V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$





Same path again. Value moves back one more step each episode:
$V(1,4) \leftarrow 0 + 0.5[0 + 0.9 \cdot 0.225 - 0] \approx 0.10$,
$V(2,4) \leftarrow 0.225 + 0.5[0 + 0.9 \cdot 0.75 - 0.225] = 0.45, \quad V(3,4) \leftarrow 0.875.$

TD(0) on GridWorld: Final values after N episodes



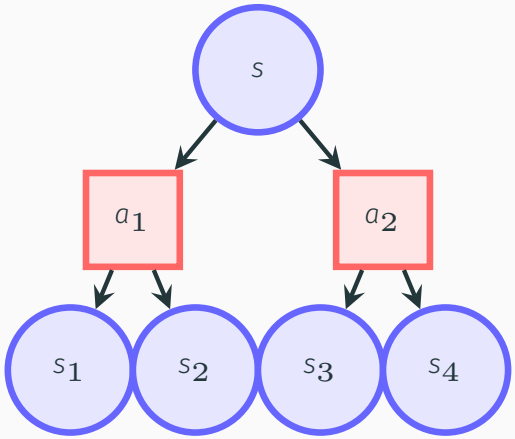
How do TD, MC, and DP compare?

Property	Dynamic Programming	Monte Carlo	Temporal Difference
Target	Expected value	G_t	$R_{t+1} + \gamma V(S_{t+1})$
Bootstraps	Yes	No	Yes
Sampling	No	Yes	Yes
Model Required	Yes	No	No
Online Learning	Yes	No	Yes
Episode Completion	Not needed	Required	Not needed

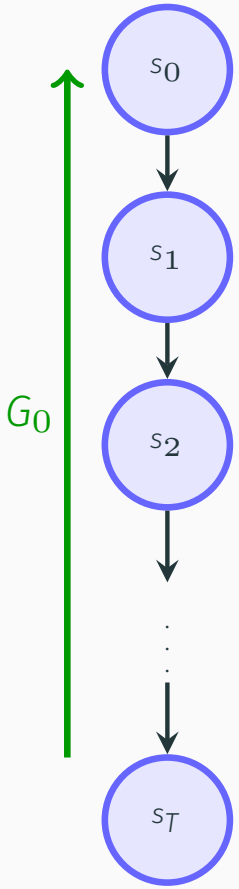
TD gets the best of both worlds!

How do the backup diagrams differ?

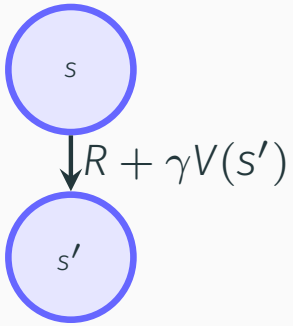
DP: Full (One-step) lookahead



MC: Complete episode



TD: One step



TD(0)

Key insight: TD uses the simplest possible backup

menti.com - Code: **2742 7364**

In the TD(0) update $V(s) \leftarrow V(s) + \alpha[r + \gamma V(s') - V(s)]$, what is the TD target?

- A) $V(s)$
- B) r
- C) $r + \gamma V(s')$
- D) $r + \gamma V(s') - V(s)$
- E) $\alpha[r + \gamma V(s') - V(s)]$

🎮 Mentimeter Poll

menti.com - Code: **2742 7364**

The main advantage of TD learning over Monte Carlo is:

- A) Lower computational complexity per update
- B) Better final performance
- C) No bias in estimates
- D) Updates before an episode ends
- E) Requires far less memory than MC in tabular settings

menti.com - Code: **2742 7364**

Why does Monte Carlo have zero bias but high variance?

- A) It averages true returns, which vary
- B) It doesn’t use bootstrapping in its updates
- C) It requires complete episodes before updating
- D) It samples from the environment
- E) It doesn’t need a model

🎮 Mentimeter Poll: All clear so far?

menti.com — Code: **2742 7364**

Clarity on TD(0), bias-variance, and differences from MC?

- A) All clear
- B) TD error concept unclear
- C) Why TD bootstraps unclear
- D) Difference from MC unclear

From Prediction to Control

1. Introduction

2. TD(0) for Policy Evaluation

3. From Prediction to Control

4. SARSA: On-Policy TD Control

5. Q-Learning: Off-Policy TD Control

6. Expected SARSA

7. Double Learning

Recall from MC: Why can't we use state values for control?

Policy evaluation (prediction) estimates V^π for given π

For control:
We need to find optimal policy π^*

Challenge:
State values $V^\pi(s)$ alone aren't enough!

With model:

$$\pi^*(s) = \text{argmax}_a \sum_{s',r} P(s', r|s, a)[r + \gamma V^*(s')]$$

Without model: We can't compute this expectation!

Solution:
Learn action values $Q^\pi(s, a)$ directly!

Recall from MC: How do we handle exploration vs exploitation?

As with Monte Carlo methods, we face the need to trade off exploration and exploitation, and again approaches fall into two main classes: **on-policy** and **off-policy**.

Recall the two approaches:

- **On-policy**: Learn about the policy we're following
- **Off-policy**: Learn about a different policy than we're following

Recall from MC: How do we handle exploration vs exploitation?

As with Monte Carlo methods, we face the need to trade off exploration and exploitation, and again approaches fall into two main classes: **on-policy** and **off-policy**.

Recall the two approaches:

- **On-policy**: Learn about the policy we're following
- **Off-policy**: Learn about a different policy than we're following

SARSA: On-Policy TD Control

1. Introduction

2. TD(0) for Policy Evaluation

3. From Prediction to Control

4. SARSA: On-Policy TD Control

5. Q-Learning: Off-Policy TD Control

6. Expected SARSA

7. Double Learning

What is SARSA and why is it called that?

Definition (SARSA Algorithm)
SARSA (State-Action-Reward-State-Action) is an on-policy TD control method:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[\underbrace{R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})}_{\text{TD target}} - Q(S_t, A_t)]$$

Key insight: We need both the current action A_t and the next action A_{t+1} chosen by our policy

How does the SARSA algorithm work?

Algorithm Structure:

1. Choose action A_t using policy derived from Q (e.g., ϵ-greedy)
2. Take action, observe R_{t+1}, S_{t+1} (i.e. env.step())
3. Choose next action A_{t+1} using same policy
4. Update: $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$
5. Move to next state-action pair: $(S_t, A_t) \leftarrow (S_{t+1}, A_{t+1})$

SARSA learns the Q-function for the policy it's following

How do we implement SARSA?

SARSA (On-policy TD Control)

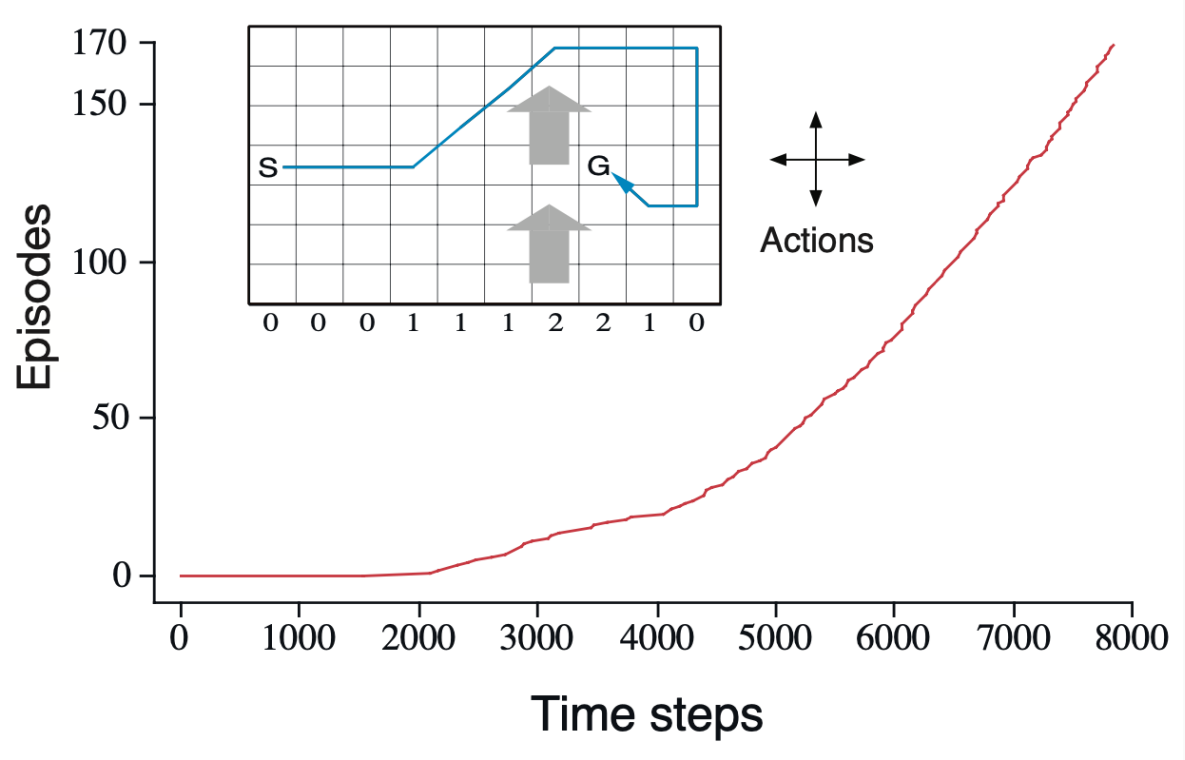
Algorithm parameter: Step size $\alpha \in (0, 1]$, small $\epsilon > 0$
Initialize: $Q(s, a)$ for all $s \in \mathcal{S}^+$, $a \in \mathcal{A}(s)$, except $Q(\text{terminal}, \cdot) = 0$

repeat
    Initialize S (i.e. env.reset())
    Choose A from S using policy derived from Q (e.g., ϵ-greedy)
    repeat for each step of episode
        Take action A, observe R, S' (i.e. env.step())
        Choose A' from S' using policy derived from Q (e.g., ϵ-greedy)
        $Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma Q(S', A') - Q(S, A)]$
        $S \leftarrow S'; A \leftarrow A'$
    until S is terminal
until convergence

Key insight: Always uses the action that will actually be taken next

Does SARSA work in practice?

Windy GridWorld Example:



Environment:

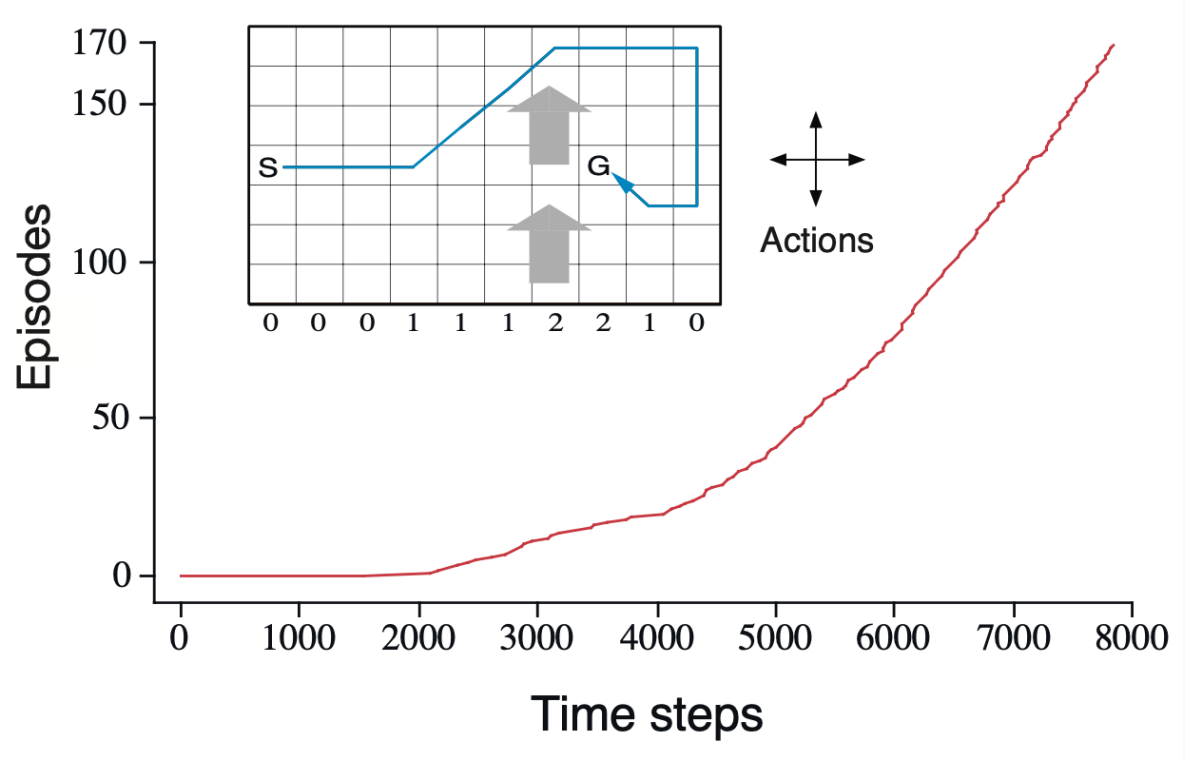
- Wind shifts agent upward
- Wind strength varies by column
- Reward: -1 per step until goal

Monte Carlo methods would struggle!

- Termination is not guaranteed for all policies.
- TD methods (like SARSA) quickly learn during the episode that some policies are poor, and switch to something else.

Does SARSA work in practice?

Windy GridWorld Example:



Environment:

- Wind shifts agent upward
- Wind strength varies by column
- Reward: -1 per step until goal

Monte Carlo methods would struggle!

- Termination is not guaranteed for all policies.
- TD methods (like SARSA) quickly learn during the episode that some policies are poor, and switch to something else.

menti.com - Code: **2742 7364**

Why is SARSA called an ‘on-policy’ method?

- A) It learns online during episodes, step by step
- B) It learns the policy directly without Q-values
- C) It updates the policy after every single step
- D) It learns about the policy it follows
- E) It only works with deterministic policies

🎮 Mentimeter Poll: All clear so far?

menti.com — Code: **2742 7364**

Clarity on SARSA?

- A) All clear
- B) Why it's called SARSA unclear
- C) On-policy concept unclear

Q-Learning: Off-Policy TD Control

1. Introduction

2. TD(0) for Policy Evaluation

3. From Prediction to Control

4. SARSA: On-Policy TD Control

5. Q-Learning: Off-Policy TD Control

6. Expected SARSA

7. Double Learning

Recall from MC: The Exploration-Exploitation Dilemma

The Exploration-Exploitation Dilemma:

- Want to learn about the **optimal** policy (deterministic, greedy)
- Need to **explore** to find all good actions (stochastic, random)
- Can't do both with the same policy!

On-policy compromise: Learn near-optimal ϵ-soft policies

Off-policy solution: Use two different policies!

- **Target policy**: What we want to learn about (can be deterministic)
- **Behavior policy**: What we use to generate data (must explore)

Recall from MC: The Exploration-Exploitation Dilemma

The Exploration-Exploitation Dilemma:

- Want to learn about the **optimal** policy (deterministic, greedy)
- Need to **explore** to find all good actions (stochastic, random)
- Can't do both with the same policy!

On-policy compromise: Learn near-optimal ϵ-soft policies

Off-policy solution: Use two different policies!

- **Target policy**: What we want to learn about (can be deterministic)
- **Behavior policy**: What we use to generate data (must explore)

Recall from MC: The Exploration-Exploitation Dilemma

The Exploration-Exploitation Dilemma:

- Want to learn about the **optimal** policy (deterministic, greedy)
- Need to **explore** to find all good actions (stochastic, random)
- Can't do both with the same policy!

On-policy compromise: Learn near-optimal ϵ-soft policies

Off-policy solution: Use two different policies!

- **Target policy**: What we want to learn about (can be deterministic)
- **Behavior policy**: What we use to generate data (must explore)

How does Q-learning solve this dilemma?

Definition (Q-Learning Algorithm)
Q-learning learns Q^* regardless of the policy followed:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)]$$

Key difference from SARSA: Uses $\max_a Q(S_{t+1}, a)$ instead of $Q(S_{t+1}, A_{t+1})$

Learns about optimal actions, not the actions actually taken!

How does Q-learning solve this dilemma?

Definition (Q-Learning Algorithm)
Q-learning learns Q^* regardless of the policy followed:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)]$$

Key difference from SARSA: Uses $\max_a Q(S_{t+1}, a)$ instead of $Q(S_{t+1}, A_{t+1})$

Learns about optimal actions, not the actions actually taken!

How do we implement Q-learning?

Q-Learning (Off-policy TD Control)

Algorithm parameter: Step size $\alpha \in (0, 1]$
Initialize: $Q(s, a)$ for all $s \in \mathcal{S}^+$, $a \in \mathcal{A}(s)$, except $Q(\text{terminal}, \cdot) = 0$

repeat
    Initialize S (i.e. env.reset())
    repeat for each step of episode
        Choose A from S using policy derived from Q (e.g., ϵ-greedy)
        Take action A, observe R, S' (i.e. env.step())
        $Q(S, A) \leftarrow Q(S, A) + \alpha[R + \gamma \max_a Q(S', a) - Q(S, A)]$
        $S \leftarrow S'$
    until S is terminal
until convergence

Notice: No need to choose next action A' - we use max instead!

Why is the max operation so powerful?

SARSA update:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

🤔 "What would happen if I follow my current policy?"

Q-learning update:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)]$$

"What would happen if I acted optimally from now on?"

Q-learning ignores exploration actions in its updates!

Why is the max operation so powerful?

SARSA update:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

🤔 "What would happen if I follow my current policy?"

Q-learning update:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)]$$

🤔 "What would happen if I acted optimally from now on?"

Q-learning ignores exploration actions in its updates!

Why is the max operation so powerful?

SARSA update:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

🤔 "What would happen if I follow my current policy?"

Q-learning update:

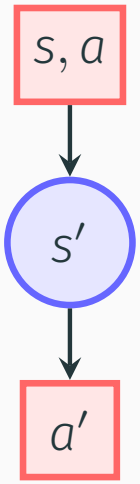
$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)]$$

🤔 "What would happen if I acted optimally from now on?"

Q-learning ignores exploration actions in its updates!

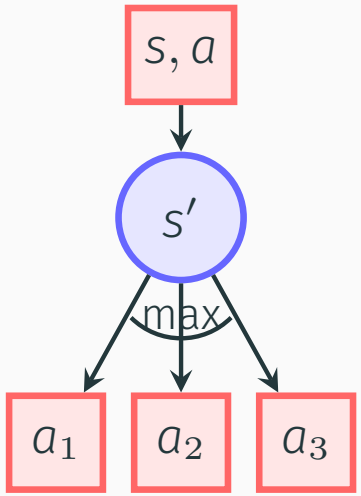
How do the backup diagrams compare?

SARSA Backup



Uses actual next action

Q-Learning Backup



Uses best possible action

Key insight: Q-learning considers all actions, takes the best

menti.com - Code: **2742 7364**

What is the key difference between SARSA and Q-learning updates?

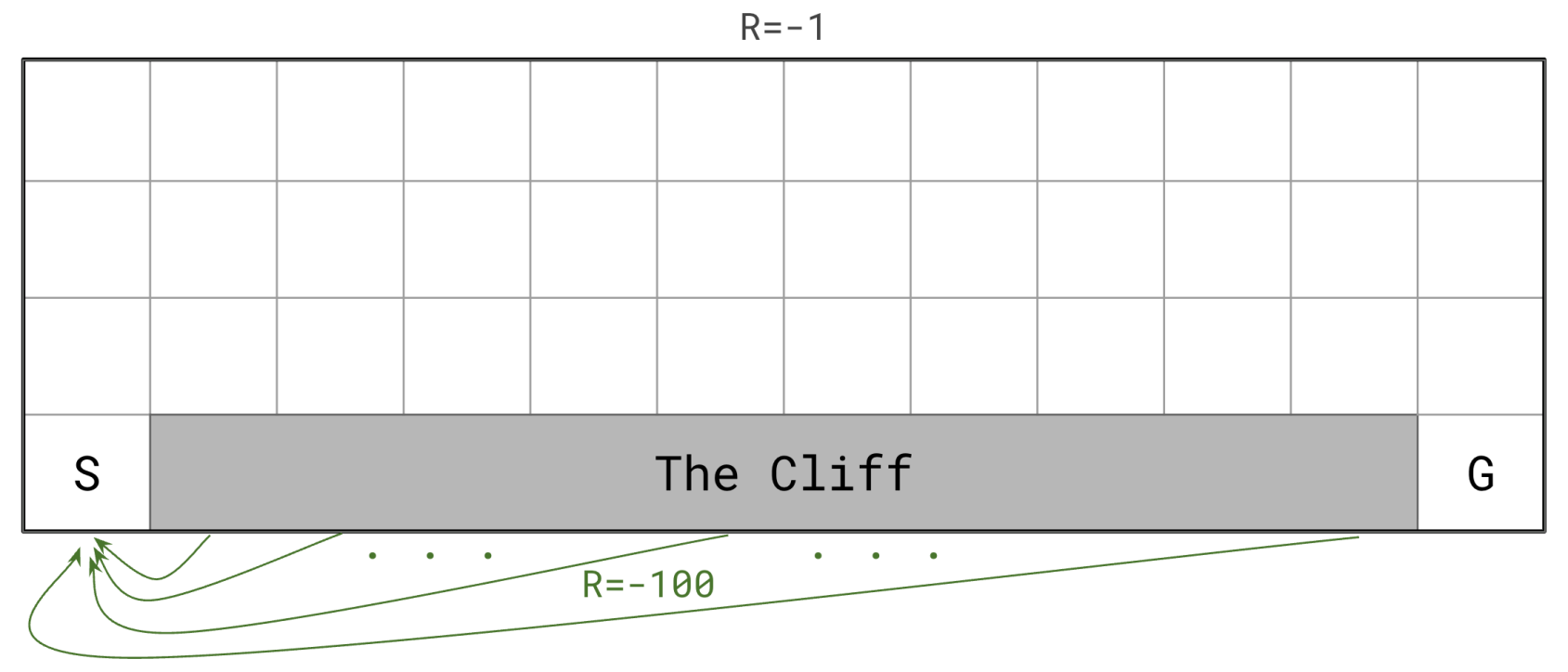
SARSA: $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma Q(s',a') - Q(s,a)]$

Q-learning: $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$

The difference is:

- A) SARSA samples a', Q-learning maximises over a'
- B) SARSA is faster than Q-learning in all settings
- C) Q-learning requires a model, SARSA doesn't
- D) SARSA works for continuing tasks, Q-learning only for episodic
- E) Q-learning has lower variance

Example: SARSA vs. Q-learning

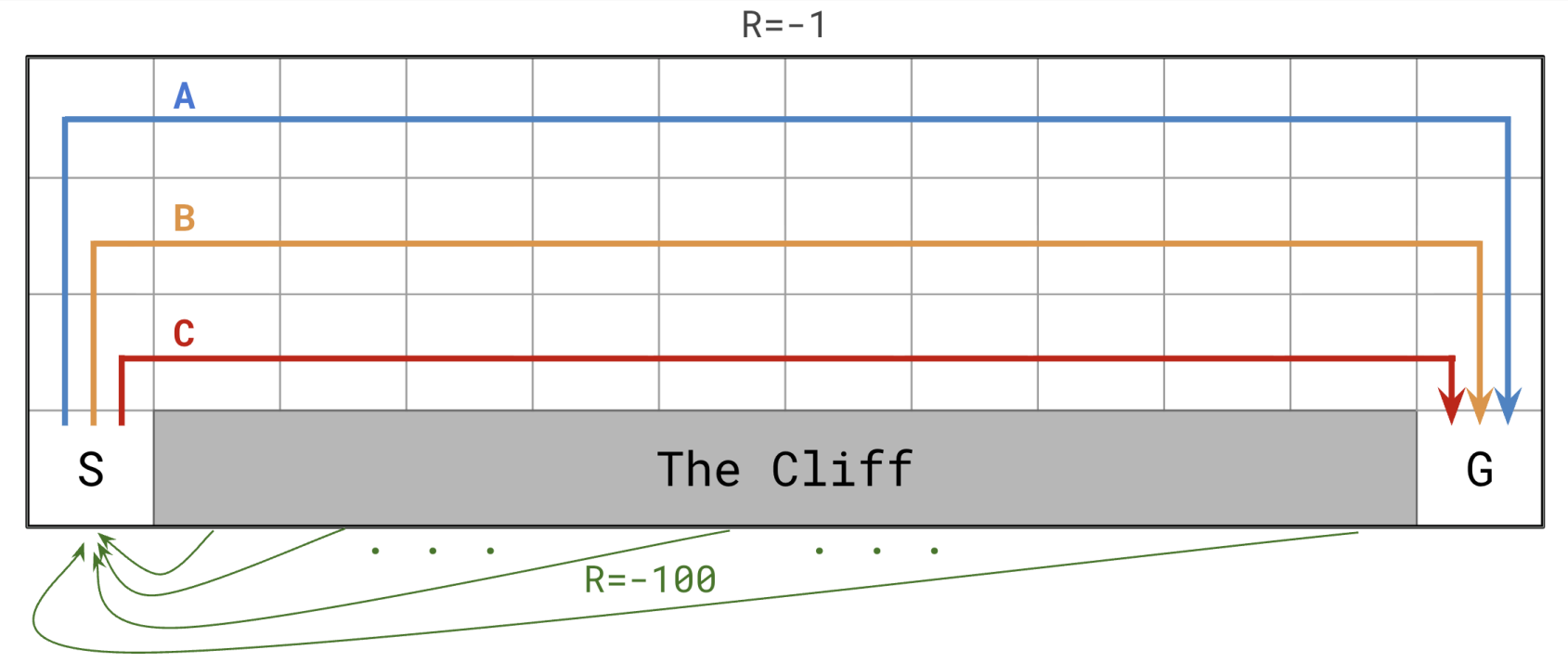


Setup:

- Cliff gives -100 reward
- Each step costs -1
- Both algorithms use ϵ-greedy with $\epsilon = 0.1$

🎮 Mentimeter Poll

menti.com - Code: **2742 7364**

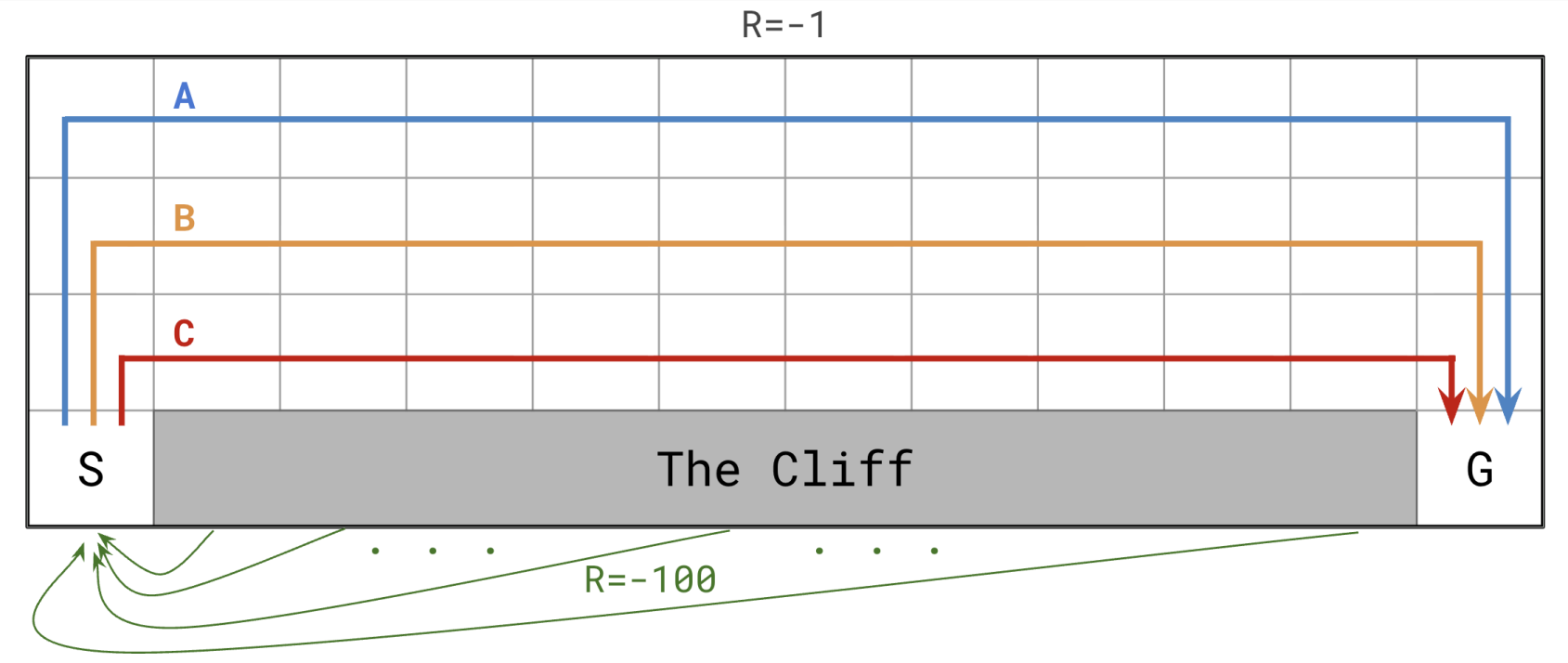


What path does SARSA learn?

- A) Path A
- B) Path B
- C) Path C

🎮 Mentimeter Poll

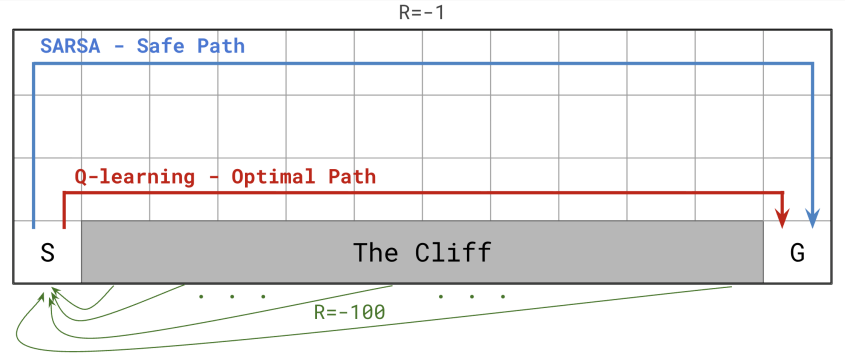
menti.com - Code: **2742 7364**

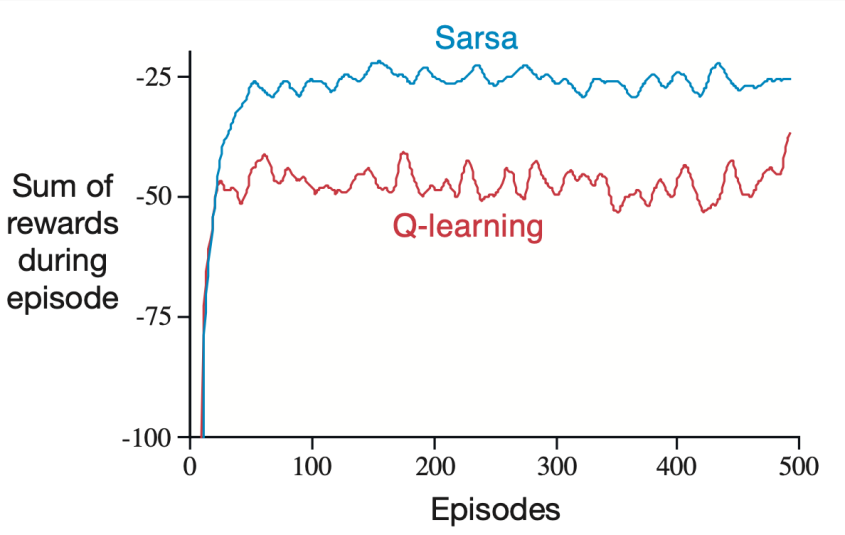


What path does Q-learning learn?

- A) Path A
- B) Path B
- C) Path C

The results: SARSA vs Q-learning paths



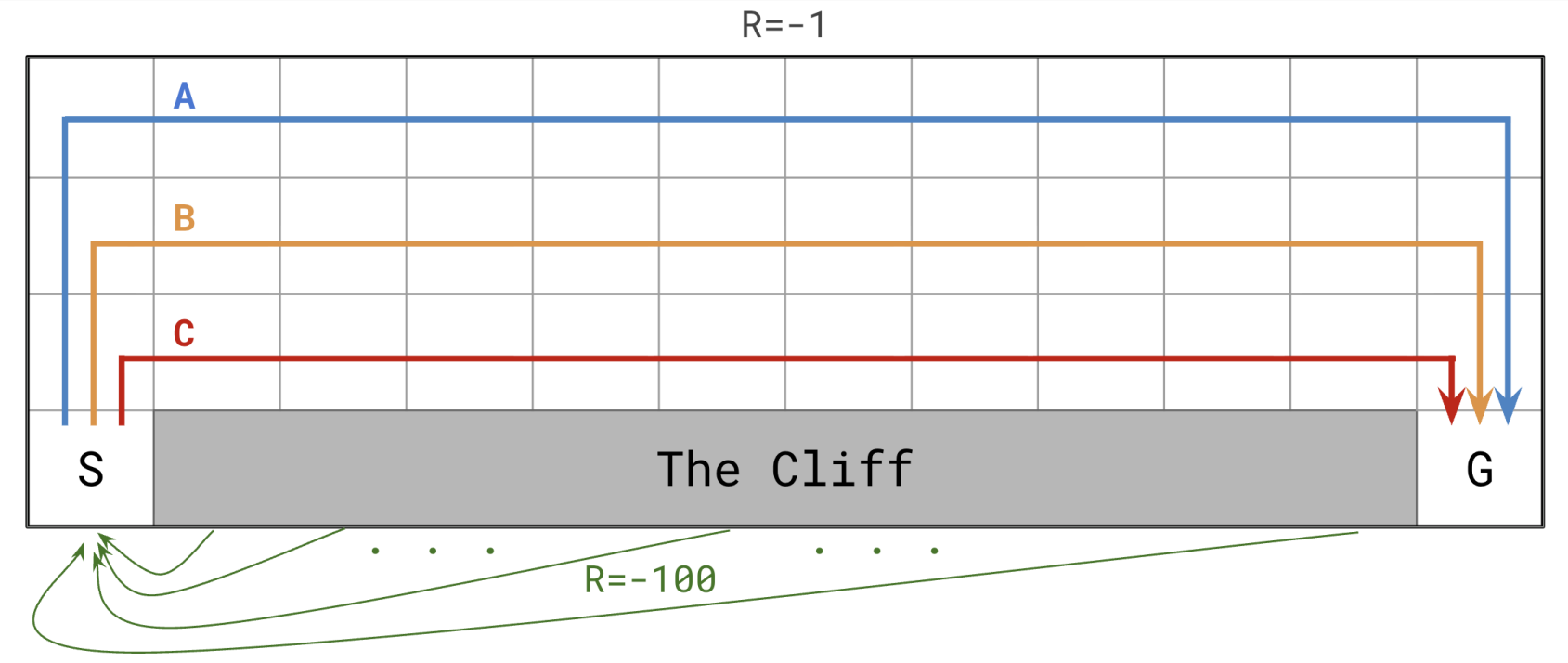


Rewards **during learning**, not the **final performance!**

Performance comparison: Q-learning gets higher final rewards but suffers during learning

🎮 Mentimeter Poll

menti.com - Code: **2742 7364**



SARSA: If we decay $\epsilon \to 0$ during learning, what path would it learn?

- A) Path A
- B) Path B
- C) Path C

🎮 Mentimeter Poll: All clear so far?

menti.com — Code: **2742 7364**

Clarity on Q-learning?

- A) All clear
- B) Why Q-learning is off-policy unclear
- C) Cliff Walking example unclear

Expected SARSA

1. Introduction

2. TD(0) for Policy Evaluation

3. From Prediction to Control

4. SARSA: On-Policy TD Control

5. Q-Learning: Off-Policy TD Control

6. Expected SARSA

7. Double Learning

Can we get the best of both SARSA and Q-learning?

The variance problem: SARSA samples an action A_{t+1}, adding noise

Definition (Expected SARSA)
Instead of sampling next action, use expected value:

$$\begin{aligned} Q(S_t, A_t) &\leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma \mathbb{E}_\pi[Q(S_{t+1}, A_{t+1})] - Q(S_t, A_t)] \\ &= Q(S_t, A_t) + \alpha[R_{t+1} + \gamma \sum_a \pi(a|S_{t+1}) Q(S_{t+1}, a) - Q(S_t, A_t)] \end{aligned}$$

Advantages:

- Removes variance from action selection
- More stable than regular SARSA
- Often faster convergence despite higher computation per step
- Can be used on-policy or off-policy

Interesting connection: When behavior is ϵ-greedy and target is greedy:

$$\text{Expected SARSA} = \text{Q-learning}$$

Can we get the best of both SARSA and Q-learning?

The variance problem: SARSA samples an action A_{t+1}, adding noise

Definition (Expected SARSA)
Instead of sampling next action, use expected value:

$$\begin{aligned} Q(S_t, A_t) &\leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma \mathbb{E}_\pi[Q(S_{t+1}, A_{t+1})] - Q(S_t, A_t)] \\ &= Q(S_t, A_t) + \alpha[R_{t+1} + \gamma \sum_a \pi(a|S_{t+1}) Q(S_{t+1}, a) - Q(S_t, A_t)] \end{aligned}$$

Advantages:

- Removes variance from action selection
- More stable than regular SARSA
- Often faster convergence despite higher computation per step
- Can be used on-policy or off-policy

Interesting connection: When behavior is ϵ-greedy and target is greedy:

$$\text{Expected SARSA} = \text{Q-learning}$$

Can we get the best of both SARSA and Q-learning?

The variance problem: SARSA samples an action A_{t+1}, adding noise

Definition (Expected SARSA)
Instead of sampling next action, use expected value:

$$\begin{aligned} Q(S_t, A_t) &\leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma \mathbb{E}_\pi[Q(S_{t+1}, A_{t+1})] - Q(S_t, A_t)] \\ &= Q(S_t, A_t) + \alpha[R_{t+1} + \gamma \sum_a \pi(a|S_{t+1}) Q(S_{t+1}, a) - Q(S_t, A_t)] \end{aligned}$$

Advantages:

- Removes variance from action selection
- More stable than regular SARSA
- Often faster convergence despite higher computation per step
- Can be used on-policy or off-policy

Interesting connection: When behavior is ϵ-greedy and target is greedy:

Expected SARSA = Q-learning

Can we get the best of both SARSA and Q-learning?

The variance problem: SARSA samples an action A_{t+1}, adding noise

Definition (Expected SARSA)
Instead of sampling next action, use expected value:

$$\begin{aligned} Q(S_t, A_t) &\leftarrow Q(S_t, A_t) + \alpha[R_{t+1} + \gamma \mathbb{E}_\pi[Q(S_{t+1}, A_{t+1})] - Q(S_t, A_t)] \\ &= Q(S_t, A_t) + \alpha[R_{t+1} + \gamma \sum_a \pi(a|S_{t+1}) Q(S_{t+1}, a) - Q(S_t, A_t)] \end{aligned}$$

Advantages:

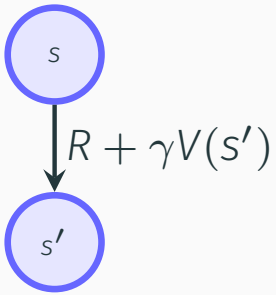
- Removes variance from action selection
- More stable than regular SARSA
- Often faster convergence despite higher computation per step
- Can be used on-policy or off-policy

Interesting connection: When behavior is ϵ-greedy and target is greedy:

$$\text{Expected SARSA} = \text{Q-learning}$$

How do all the backup diagrams compare?

TD(0)



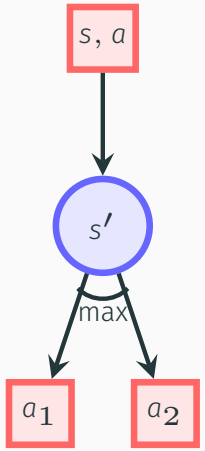
State values

SARSA



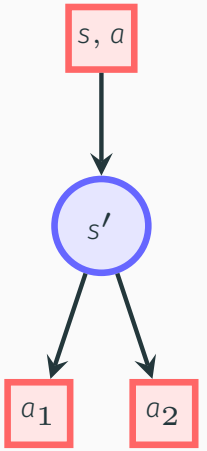
Actual A_{t+1}

Q-Learning



$\max_a Q(s', a)$

Expected SARSA



$\mathbb{E}[Q(s', A')]$

All bootstrap, but differ in action selection approach

menti.com - Code: **2742 7364**

How does Expected SARSA differ from regular SARSA?

- A) It requires a model of the environment
- B) It learns faster but is less accurate in the long run
- C) It averages over next actions
- D) It only works with deterministic policies
- E) It is an off-policy method

🎮 Mentimeter Poll: All clear so far?

menti.com — Code: **2742 7364**

Clarity on Expected SARSA and comparison of TD methods?

- A) All clear
- B) How Expected SARSA computes expectation unclear
- C) Relationship between SARSA/Q-learning/Expected SARSA unclear
- D) Backup diagram differences unclear

Double Learning

1. Introduction

2. TD(0) for Policy Evaluation

3. From Prediction to Control

4. SARSA: On-Policy TD Control

5. Q-Learning: Off-Policy TD Control

6. Expected SARSA

7. Double Learning

Q-learning: What's wrong with taking the maximum?

The maximization bias problem: $\max$ operations can be overly optimistic!

Simple example: All true Q-values are 0, but estimates are noisy

- True values: $Q^*(s, a_1) = Q^*(s, a_2) = 0$
- Estimates: $Q(s, a_1) = 0.1$, $Q(s, a_2) = -0.1$ (noise!)
- $\max(\text{estimates}) = 0.1 > 0 = \max(\text{true values})$

The problem: $\mathbb{E}[\max(\text{noisy estimates})] > \max(\text{true values})$

Result: Leads to overly optimistic policies that choose poor actions

Q-learning: What's wrong with taking the maximum?

The maximization bias problem: $\max$ operations can be overly optimistic!

Simple example: All true Q-values are 0, but estimates are noisy

- True values: $Q^*(s, a_1) = Q^*(s, a_2) = 0$
- Estimates: $Q(s, a_1) = 0.1$, $Q(s, a_2) = -0.1$ (noise!)
- $\max(\text{estimates}) = 0.1 > 0 = \max(\text{true values})$

The problem: $\mathbb{E}[\max(\text{noisy estimates})] > \max(\text{true values})$

Result: Leads to overly optimistic policies that choose poor actions

Q-learning: What's wrong with taking the maximum?

The maximization bias problem: $\max$ operations can be overly optimistic!

Simple example: All true Q-values are 0, but estimates are noisy

- True values: $Q^*(s, a_1) = Q^*(s, a_2) = 0$
- Estimates: $Q(s, a_1) = 0.1$, $Q(s, a_2) = -0.1$ (noise!)
- $\max(\text{estimates}) = 0.1 > 0 = \max(\text{true values})$

The problem: $\mathbb{E}[\max(\text{noisy estimates})] > \max(\text{true values})$

Result: Leads to overly optimistic policies that choose poor actions

How does double learning solve maximization bias?

Key idea: Separate action selection from action evaluation

Double learning approach:

- Maintain **two independent estimates** Q_1 and Q_2
- Use Q_1 to select action: $A^* = \text{argmax}_a Q_1(s, a)$
- Use Q_2 to evaluate action: $Q_2(s, A^*)$

Selection and evaluation are independent, eliminating bias!

Can also repeat the process with the role of the two estimates reversed to yield a second unbiased estimate $Q_1(s, \text{argmax}_a Q_2(s, a))$,

How does double learning solve maximization bias?

Key idea: Separate action selection from action evaluation

Double learning approach:

- Maintain **two independent estimates** Q_1 and Q_2
- Use Q_1 to select action: $A^* = \text{argmax}_a Q_1(s, a)$
- Use Q_2 to evaluate action: $Q_2(s, A^*)$

Selection and evaluation are independent, eliminating bias!

Can also repeat the process with the role of the two estimates reversed to yield a second unbiased estimate $Q_1(s, \text{argmax}_a Q_2(s, a))$,

How do we implement Double Q-learning?

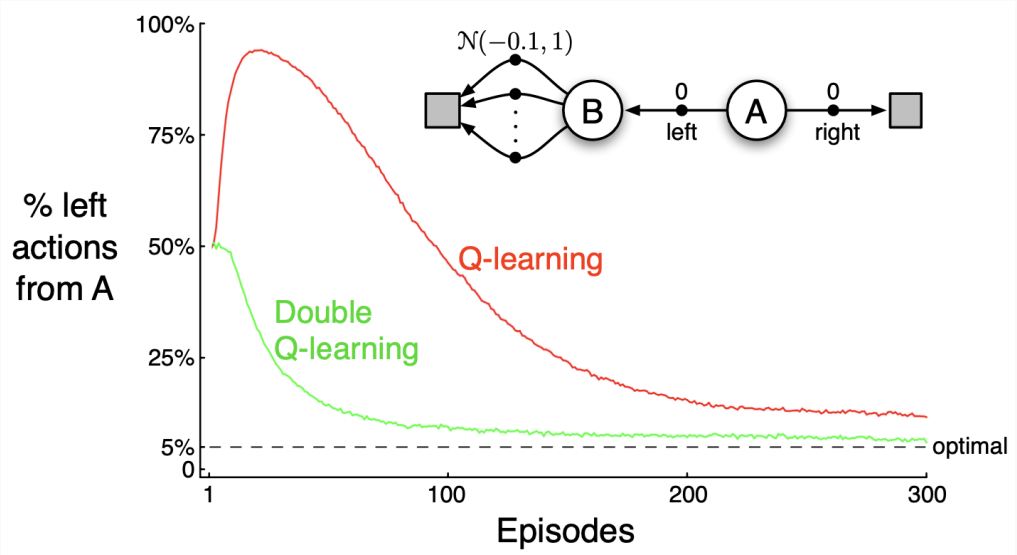
Double Q-Learning (Off-policy TD Control)

Algorithm parameters: Step size $\alpha \in (0, 1]$, small $\epsilon > 0$
Initialize: $Q_1(s, a)$ and $Q_2(s, a)$ for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, except $Q_1(\text{terminal}, \cdot) = 0$ and $Q_2(\text{terminal}, \cdot) = 0$

repeat
    Initialize S (i.e. env.reset())
    repeat for each step of episode
        Choose A from S using policy ϵ-greedy in $Q_1 + Q_2$
        Take action A, observe R, S' (i.e. env.step())
        if With 0.5 probability: **then**
            $Q_1(S, A) \leftarrow Q_1(S, A) + \alpha[R + \gamma Q_2(S', \text{argmax}_a Q_1(S', a)) - Q_1(S, A)]$
        else
            $Q_2(S, A) \leftarrow Q_2(S, A) + \alpha[R + \gamma Q_1(S', \text{argmax}_a Q_2(S', a)) - Q_2(S, A)]$
        end if
        $S \leftarrow S'$
    until S is terminal
until convergence

Cost: Double memory, but much more robust learning

Does double learning actually work?



The MDP Setup:

- **State A**: Choose left (go to B) or right (terminate with reward 0)
- **State B**: Many actions, each terminates with reward $\sim \mathcal{N}(-0.1, 1.0)$
- **Optimal policy**: Always go right from A (expected return = 0)
- **The trap**: Going left has expected return = -0.1; maximization bias makes B look positive (due to noise)

Q-learning's max operation over noisy estimates creates false optimism about state B

🎮 Mentimeter Poll

menti.com - Code: **2742 7364**

What causes maximization bias in Q-learning?

- A) Using ϵ-greedy exploration
- B) Insufficient exploration
- C) Bootstrapping from current estimates
- D) Learning off-policy from a behaviour policy
- E) Maximising over noisy estimates

Recap: What are TD methods' main advantages?

Advantages:

- **Online learning**: Update after every step
- **Model-free**: No environment model required
- **Computational efficiency**: Simple, fast updates
- **Flexibility**: Work with continuing tasks
- **Lower variance**: Bootstrapping reduces variance vs MC
- **Faster learning**: Often converges faster than MC

TD methods are the foundation of modern deep RL!

Recap: What are the key limitations/challenges we should know?

Limitations:

- **Biased estimates**: Due to bootstrapping from imperfect estimates
- **Sensitive to initialization**: Poor initial values slow learning
- **Maximization bias**: In methods using max operations
- **Exploration challenges**: Still need good exploration strategies

What have we learned today?

Key Takeaways

- **TD Learning**: Combines sampling (MC) with bootstrapping (DP)
- **TD(0)**: Simplest TD method for policy evaluation
- **SARSA**: On-policy TD control - safe but conservative
- **Q-Learning**: Off-policy TD control - learns optimal policy directly
- **Expected SARSA**: Reduces variance through expectation
- **Double Learning**: Addresses maximization bias

Next Week: Value Function Approximation

- From tabular to function approximation
- Neural networks
- Foundation for Deep Q-Networks (DQN)

What have we learned today?

Key Takeaways

- **TD Learning**: Combines sampling (MC) with bootstrapping (DP)
- **TD(0)**: Simplest TD method for policy evaluation
- **SARSA**: On-policy TD control - safe but conservative
- **Q-Learning**: Off-policy TD control - learns optimal policy directly
- **Expected SARSA**: Reduces variance through expectation
- **Double Learning**: Addresses maximization bias

Next Week: Value Function Approximation

- From tabular to function approximation
- Neural networks
- Foundation for Deep Q-Networks (DQN)

Reading and Resources

📚 Essential Reading

- **Sutton & Barto**: Chapter 6 (Temporal-Difference Learning)
- Focus on: TD Prediction (6.1), Advantages of TD (6.2), Optimality of TD(0) (6.3)
- Important: SARSA (6.4), Q-learning (6.5), Expected SARSA (6.6)
- Advanced: Double Learning (6.7)

🎮 Lecture Feedback

menti.com - Code: **2742 7364**

Q1: If you got lost today, at what point did you lose track?

- A) TD(0) and TD error
- B) Bias-variance trade-off
- C) SARSA
- D) Q-learning
- E) Expected SARSA
- F) Double learning and maximization bias
- G) None of the above, I followed everything

Q2: Generic feedback - what worked well, what could be improved?